Estrutura de Dados II

Prof. Vanessa Matias Leite

1

Heaps e Listas de Prioridade

2

Listas de Prioridades

Qualquer problema em que seja preciso estabelecer uma prioridade de acesso aos elementos pode ser representado com uma lista de prioridade

- a fila de prioridade do processador;
- uma fila de pacientes esperando transplante de fígado;
- busca em grafos (algoritmo de Dijkstra);
- compressão de dados (código de Huffman);
- sistemas operacionais (manipulação de interrupções);
- fila de pouso de aviões em um aeroporto (combustível disponível).
- Fila de impressora

3

Listas de Prioridades

- A prioridade associada a um dado pode ser qualquer coisa:
  - tempo, custo, etc, mas precisa ser um **escalar.**
- Cuidar da eficiência
  - Seleção do elemento com maior (ou menor) prioridade
  - Remoção do elemento de maior (ou menor) prioridade
  - inserção de um novo elemento

4

Implementações de Lista de Prioridade

- Listas não ordenadas;
- Lista Ordenadas;
- Heaps;

5

Implementações de Lista de Prioridade

Lista não ordenada:

- ✓ Inserção e Construção: elementos (registros) podem ser colocados na tabela em qualquer ordem
- ✓ Remoção: percorrer a tabela sequencialmente em busca do elemento de maior prioridade
- ✓ Alteração: não implica em mudança na estrutura da tabela, mas exige busca do elemento a ser alterado
- ✓ Seleção: idem à Remoção

6

Implementações de Lista de Prioridade

Lista ordenada:

- ✓ Remoção e Seleção: imediata, pois elemento de maior prioridade é o primeiro da tabela
- ✓ Inserção: exige percorrer a tabela para encontrar a posição correta de inserção
- ✓ Alteração: semelhante a uma nova inserção
- ✓ Construção: exige ordenação prévia da tabela

7

Listas de Prioridades

Operação	Lista	Lista ordenada	Árvore balanceada	Heap
Seleção	O(n)	O(1)	O(log n)	O(1)
Inserção	O(1)	O(n)	O(log n)	O(log n)
Remoção (do menor)	O(n)	O(1)	O(log n)	O(log n)
Alteração de prioridade	O(n)	O(n)	O(log n)	O(log n)
Construção	O(n)	O(n log n)	O(n log n)	O(n)

8

Heap

- Lista linear (vetor) composta de elementos com chaves si, ..., s_n,
- Chaves representam as prioridades
- Não existem dois elementos com a mesma prioridade

- Heap máximo: chaves $s_1,\ldots, s_n$, tal que $s_i \leq s_{i/2}$para $1<i\leq n$ (cada nó é maior que seus filhos)
- Heap mínimo: chaves $s_1,\ldots, s_n$, tal que $s_i \geq s_{i/2}$ para $1<i \leq n$ (cada nó é menor que seus filho)

9

Heap

- A **relação entre as chaves** pode ser **MODELADA** por árvores binárias

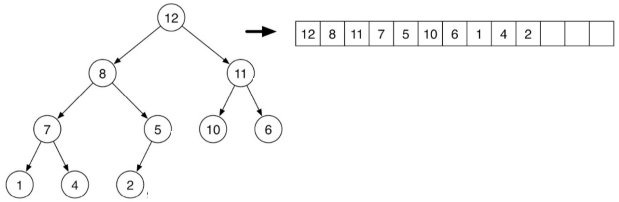
- A raiz contém a chave de maior prioridade (heap máximo)
  - este é elemento 1 da lista

- Cada nó possui prioridade **maior** do que seus dois filhos

- Heaps são implementados usando **vetores ou listas sequenciais.**

10

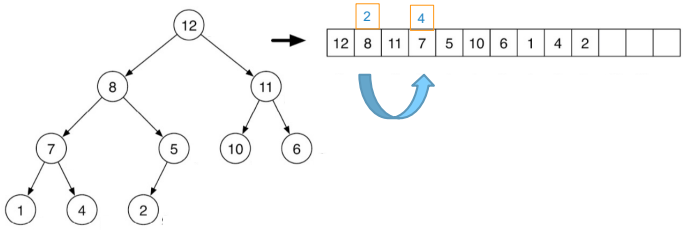
Heap



11

Heap

- **Heap máximo**: chaves $s_1,\ldots, s_n$, tal que $s_i < s_{i/2}$ para $1<i\leq n$



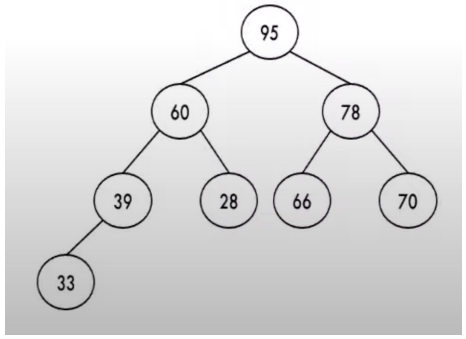
12

Heap - Propriedades

Para um determinado elemento i:

- filho esquerdo de i : 2i
- filho direita de i : 2i + 1
- nó pai de i : i / 2

1	2	3	4	5	6	7	8
95	60	78	39	28	66	70	33



13

Heap - Propriedades

```
int pai(int i){
    return (i/2);
}

int esq(int i){
    return (i*2);
}

int dir(int i){
    return (i*2+1);
}
```

14

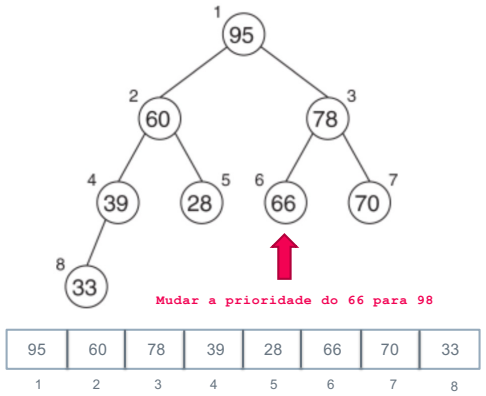
Heap – Alteração de Prioridade

As chaves podem alterar as suas prioridades, por este motivo é necessário reorganizar o heap:

- Um nó que tem prioridade aumentada precisa “subir” na árvore;
- Um nó que tem prioridade diminuída precisa “descer” na árvore;

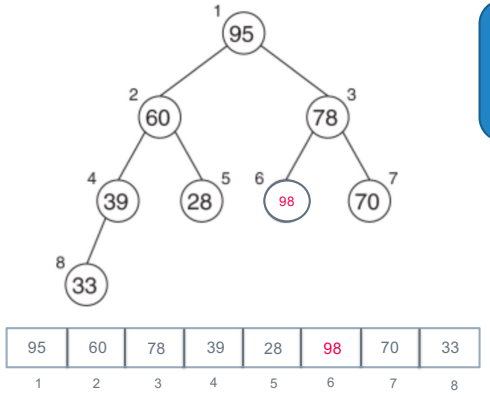
15

Heap – Alteração de Prioridade



16

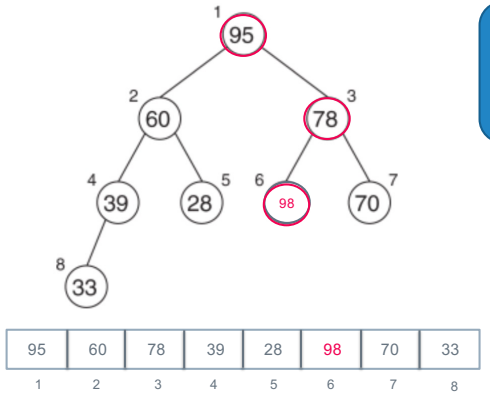
Heap – Alteração de Prioridade



Apenas o ramo que vai do nó atualizado até a raiz é afetado!

17

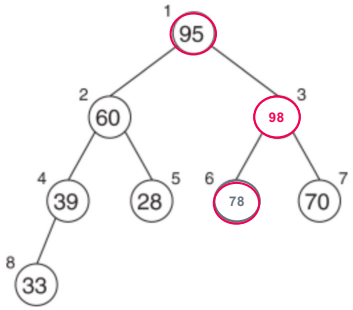
Heaps – Alteração de Prioridade



Subimos o nó, fazendo trocas com o nó pai, até que a árvore volte a ficar correta!

18

Heap – Alteração de Prioridade

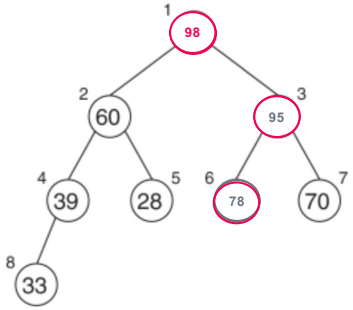


95	60	98	39	28	78	70	33
1	2	3	4	5	6	7	8

Subimos o nó, fazendo trocas com o nó pai, até que a árvore volte a ficar correta!

19

Heap – Alteração de Prioridade



98	60	95	39	28	78	70	33
1	2	3	4	5	6	7	8

Subimos o nó, fazendo trocas com o nó pai, até que a árvore volte a ficar correta!

20

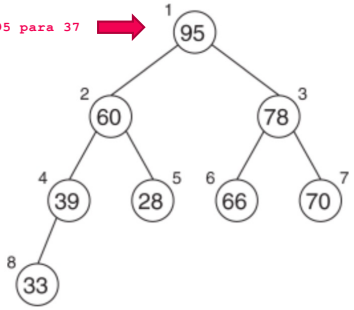
Heap – Alteração de Prioridade

```
void subir(int *heap, int i) {
    int j = pai(i);
    if (j >= 1) {
        if (heap[i] > heap[j]) {
            //faz a subida
            int temp = heap[i];
            heap[i] = heap[j];
            heap[j] = temp;
            subir(heap, j);
        }
    }
}
```

21

Heap – Alteração de Prioridade

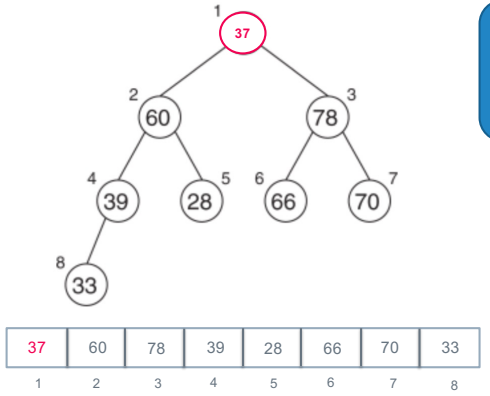
Mudar a prioridade do 95 para 37



95	60	78	39	28	66	70	33
1	2	3	4	5	6	7	8

22

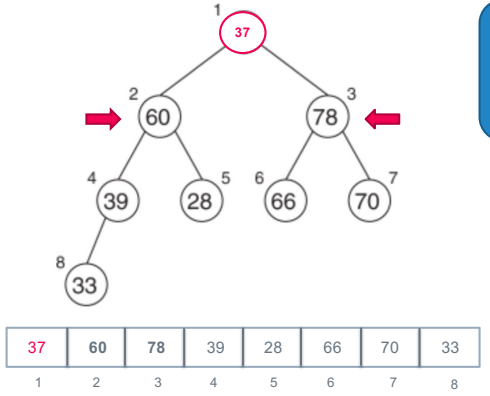
Heap – Alteração de Prioridade



"Descer" o elemento alterado, fazendo trocas com o nó filho de maior prioridade.

23

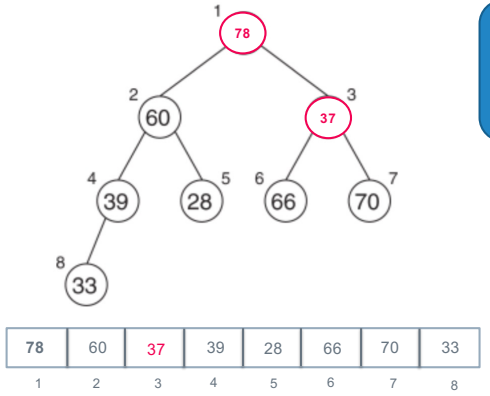
Heap – Alteração de Prioridade



"Descer" o elemento alterado, fazendo trocas com o nó filho de maior prioridade.

24

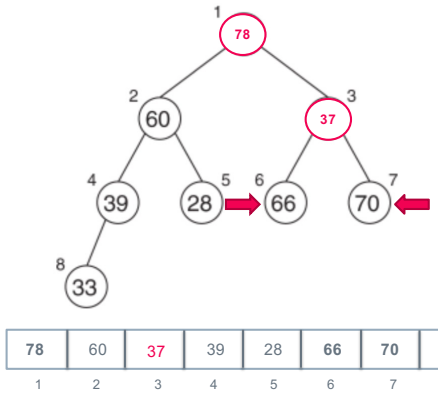
Heap – Alteração de Prioridade



"Descer" o elemento alterado, fazendo trocas com o nó filho de maior prioridade.

25

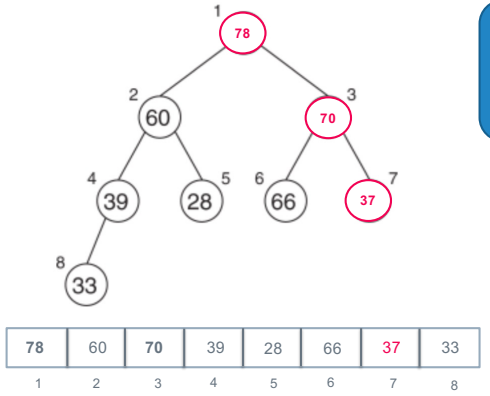
Heap – Alteração de Prioridade



"Descer" o elemento alterado, fazendo trocas com o nó filho de maior prioridade.

26

Heap – Alteração de Prioridade



"Descer" o elemento alterado, fazendo trocas com o nó filho de maior prioridade.

27

Heap – Alteração de Prioridade

```
void descer(int *heap, int i, int n){
    //descobre quem é o maior filho de i
    int e = esq(i);
    int d = dir(i);
    int maior = i;
    if (e<=n && heap[e] > heap[i]) {
        maior=e;
    }
    if (d<=n && heap[d] > heap[maior]) {
        maior=d;
    }
    if (maior != i){
        //faz a descida trocando com o maior filho
        int temp=heap[i];
        heap[i] = heap[maior];
        heap[maior]=temp;
        descer(heap, maior, n);
    }
}
```

28

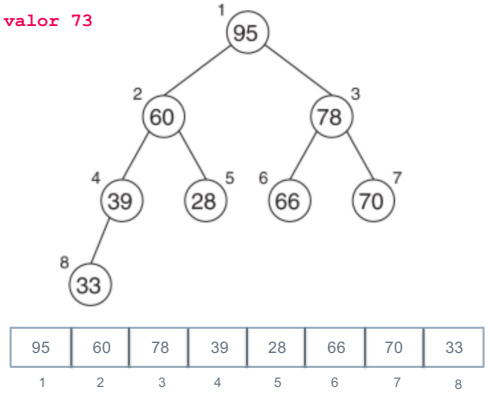
Heap - Inserção

- Vetor com m elementos
- Inserir novo elemento na posição m+1 do vetor
- Assumir que esse elemento já existia e teve sua **prioridade aumentada**
- Executar algoritmo de **subida** na árvore para corrigir a prioridade e colocar o novo elemento na posição correta

29

Heap - Inserção

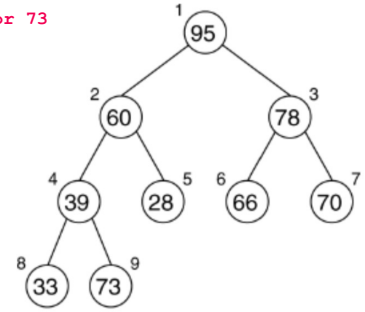
Inserir valor 73



30

Heap - Inserção

Inserir valor 73

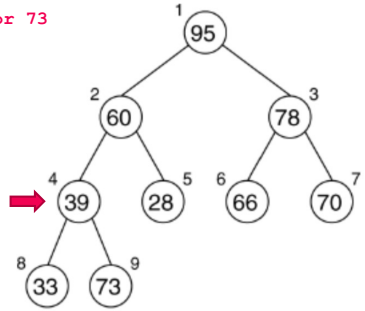


95	60	78	39	28	66	70	33	73
1	2	3	4	5	6	7	8	9

31

Heap - Inserção

Inserir valor 73

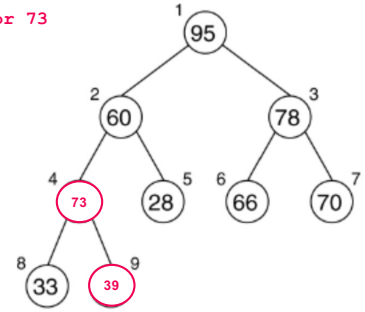


95	60	78	**39**	28	66	70	33	73
1	2	3	4	5	6	7	8	9

32

Heap - Inserção

Inserir valor 73

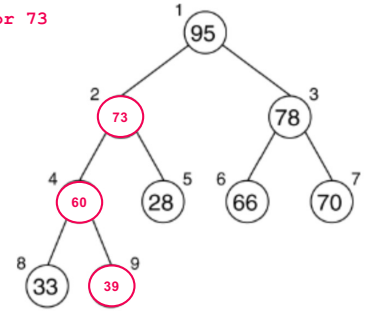


1	2	3	4	5	6	7	8	9
95	60	78	73	28	66	70	33	39

33

Heap - Inserção

Inserir valor 73



1	2	3	4	5	6	7	8	9
95	73	78	60	28	66	70	33	39

34

Heap - Inserção

```
int insere(int *heap, int novo, int n) {
    //aumenta o tamanho do vetor
    heap = (int *) realloc(heap, sizeof(int) * (n + 2));
    n = n + 1;
    heap[n] = novo;

    subir(heap, n);
    //retorna o novo valor de n
    return n;
}
```

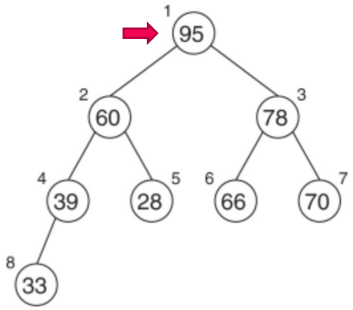
35

Heap – Remoção

- Remover o primeiro elemento da tabela
- Preencher o espaço vazio deixado por ele com o último elemento da tabela
- Executar o algoritmo de descida na árvore para corrigir a prioridade desse elemento

36

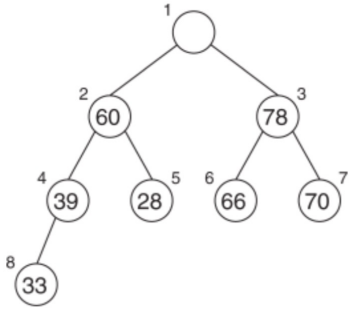
Heap - Remoção



95	60	78	39	28	66	70	33
1	2	3	4	5	6	7	8

37

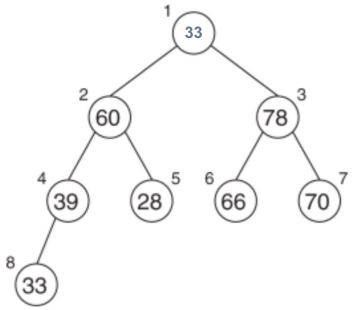
Heap - Remoção



	60	78	39	28	66	70	33
1	2	3	4	5	6	7	8

38

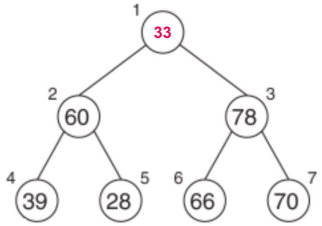
Heap - Remoção



1	2	3	4	5	6	7	8
33	60	78	39	28	66	70	

39

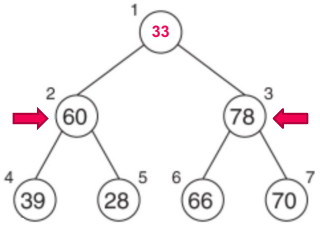
Heap - Remoção



1	2	3	4	5	6	7
33	60	78	39	28	66	70

40

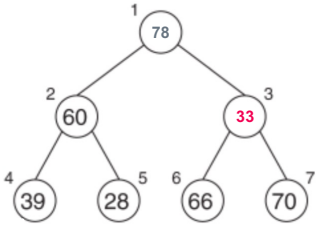
Heap - Remoção



33	60	78	39	28	66	70
1	2	3	4	5	6	7

41

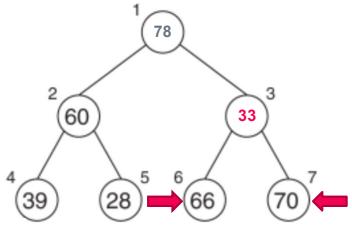
Heap - Remoção



78	60	33	39	28	66	70
1	2	3	4	5	6	7

42

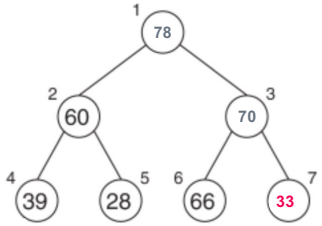
Heap - Remoção



1	2	3	4	5	6	7
78	60	33	39	28	66	70

43

Heap - Remoção



1	2	3	4	5	6	7
78	60	70	39	28	66	33

44

Heap - Remoção

```
int exclui(int *heap, int n) {
    heap[1] = heap[n];
    n = n - 1;
    //diminui o tamanho do vetor
    heap = (int *) realloc(heap, sizeof(int) * (n + 1));

    descer(heap, 1, n);
    //retorna o novo valor de n
    return n;
}
```

45

Construção de uma lista de prioridade

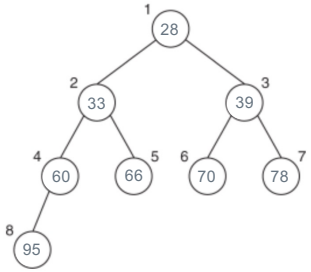
Dada uma lista L de elementos para a qual se deseja construir uma heap H, há duas alternativas:

1) Considerar uma heap vazia e ir inserindo os elementos de L um a um em H;
2) Considerar que a lista L é uma heap e corrigir as prioridades.
   - ✓ Assumir que as propriedades das folhas estão corretas (pois não tem filhos, então satisfazem à propriedade de terem prioridade maior que seus filhos).
   - ✓ Acertar as prioridades dos nós internos realizando descidas quando necessário.

46

Construção de uma lista de prioridade

Construir uma Heap a partir da lista 28, 33, 39, 60, 66, 70, 78, 95

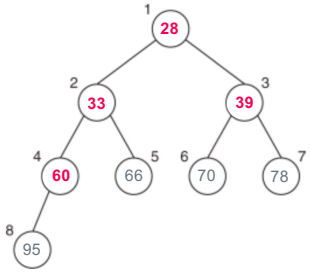


28	33	39	60	66	70	78	95
1	2	3	4	5	6	7	8

47

Construção de uma lista de prioridade

Construir uma Heap a partir da lista 28, 33, 39, 60, 66, 70, 78, 95



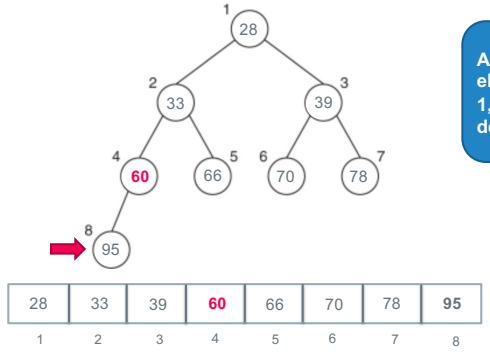
28	33	39	60	66	70	78	95
1	2	3	4	5	6	7	8

Acertar a prioridade do elemento n/2 até o elemento 1, nessa ordem, realizando descidas quando necessário

48

Construção de uma lista de prioridade

Construir uma Heap a partir da lista 28, 33, 39, 60, 66, 70, 78, 95

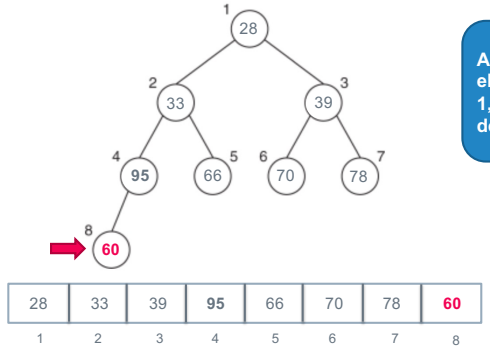


Acertar a prioridade do elemento n/2 até o elemento 1, nessa ordem, realizando descidas quando necessário

49

Construção de uma lista de prioridade

Construir uma Heap a partir da lista 28, 33, 39, 60, 66, 70, 78, 95

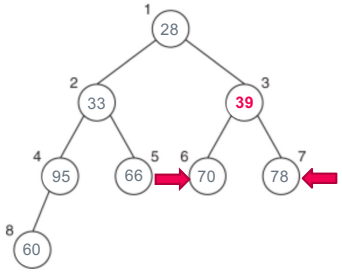


Acertar a prioridade do elemento n/2 até o elemento 1, nessa ordem, realizando descidas quando necessário

50

Construção de uma lista de prioridade

Construir uma Heap a partir da lista 28, 33, 39, 60, 66, 70, 78, 95



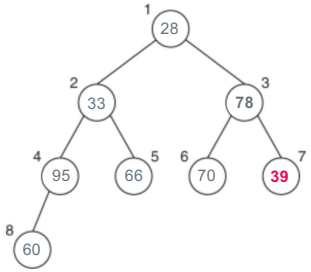
Acertar a prioridade do elemento n/2 até o elemento 1, nessa ordem, realizando descidas quando necessário

28	33	39	95	66	70	78	60
1	2	3	4	5	6	7	8

51

Construção de uma lista de prioridade

Construir uma Heap a partir da lista 28, 33, 39, 60, 66, 70, 78, 95



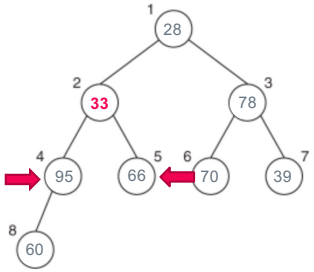
Acertar a prioridade do elemento n/2 até o elemento 1, nessa ordem, realizando descidas quando necessário

28	33	78	95	66	70	39	60
1	2	3	4	5	6	7	8

52

Construção de uma lista de prioridade

Construir uma Heap a partir da lista 28, 33, 39, 60, 66, 70, 78, 95



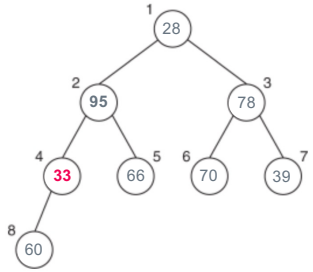
Acertar a prioridade do elemento n/2 até o elemento 1, nessa ordem, realizando descidas quando necessário

28	**33**	78	**95**	**66**	70	39	60
1	2	3	4	5	6	7	8

53

Construção de uma lista de prioridade

Construir uma Heap a partir da lista 28, 33, 39, 60, 66, 70, 78, 95



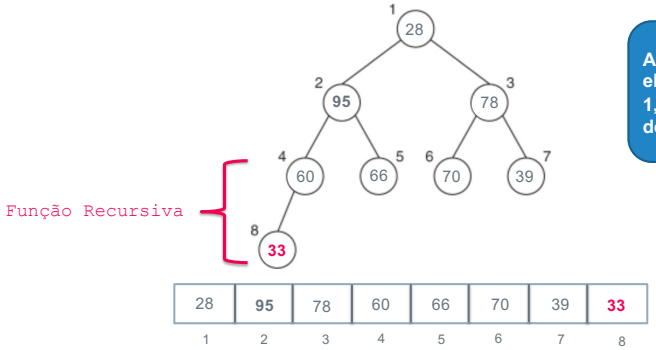
Acertar a prioridade do elemento n/2 até o elemento 1, nessa ordem, realizando descidas quando necessário

28	**95**	78	**33**	66	70	39	60
1	2	3	4	5	6	7	8

54

Construção de uma lista de prioridade

Construir uma Heap a partir da lista 28, 33, 39, 60, 66, 70, 78, 95

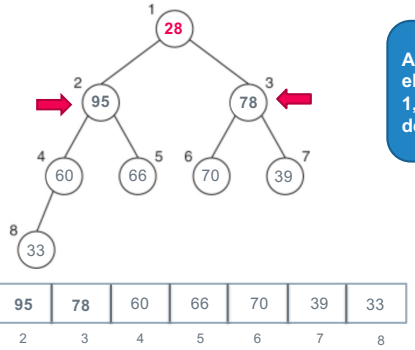


Acertar a prioridade do elemento n/2 até o elemento 1, nessa ordem, realizando descidas quando necessário

55

Construção de uma lista de prioridade

Construir uma Heap a partir da lista 28, 33, 39, 60, 66, 70, 78, 95

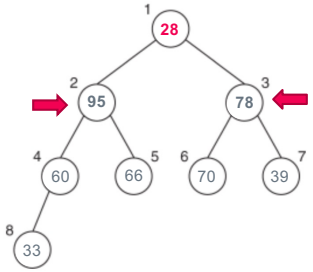


Acertar a prioridade do elemento n/2 até o elemento 1, nessa ordem, realizando descidas quando necessário

56

Construção de uma lista de prioridade

Construir uma Heap a partir da lista 28, 33, 39, 60, 66, 70, 78, 95



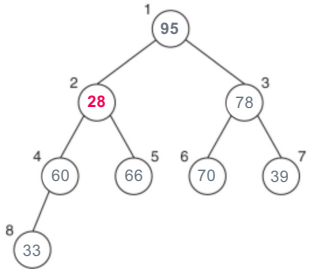
Acertar a prioridade do elemento n/2 até o elemento 1, nessa ordem, realizando descidas quando necessário

1	2	3	4	5	6	7	8
28	95	78	60	66	70	39	33

57

Construção de uma lista de prioridade

Construir uma Heap a partir da lista 28, 33, 39, 60, 66, 70, 78, 95



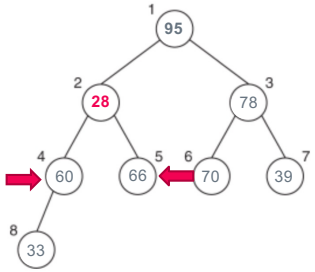
Acertar a prioridade do elemento n/2 até o elemento 1, nessa ordem, realizando descidas quando necessário

1	2	3	4	5	6	7	8
95	28	78	60	66	70	39	33

58

Construção de uma lista de prioridade

Construir uma Heap a partir da lista 28, 33, 39, 60, 66, 70, 78, 95



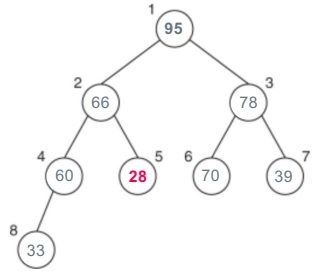
1	2	3	4	5	6	7	8
95	28	78	60	66	70	39	33

Acertar a prioridade do elemento n/2 até o elemento 1, nessa ordem, realizando descidas quando necessário

59

Construção de uma lista de prioridade

Construir uma Heap a partir da lista 28, 33, 39, 60, 66, 70, 78, 95



1	2	3	4	5	6	7	8
95	66	78	60	28	70	39	33

Acertar a prioridade do elemento n/2 até o elemento 1, nessa ordem, realizando descidas quando necessário

60

Construção de uma lista de prioridade

```
void constroi_heap_maximo(int *heap, int n){
    int i;
    int j=(n/2);
    for(i=j;i>=1;i--)
        descer(heap, i, n);
}
```

61

HeapSort

- A partir da heap, é possível ordenar os dados, fazendo trocas:
  - ✓ O maior elemento (raiz) é trocado com o último elemento do vetor
  - ✓ Esse elemento então já está na posição correta (ordenação crescente)
  - ✓ Considerar que vetor tem tamanho n-1 e descer a raiz para que o heap fique consistente
  - ✓ Repetir esses passos n-1 vezes
- A Complexidade do heap sort é O(n log n)

62